

Selective Inference with Request-Conditioned Dependency Gates for CAR-bench Track 2

Lalit Chandra Routhu

rlalitchandra99@gmail.com

Abstract

CAR-bench Track 2 evaluates reliable tool-using agents under a maximum sequential depth of five and an average budget of 500k tokens per task. We find that planner, verifier, and best-of-three inference increase compute without improving stable task success. We therefore separate structural capability detection from semantic action generation. Before each agent-generation step, a deterministic dependency gate checks whether the requested operation is supported by the live tool schema and whether required observed values are available. Unsupported requests receive a transparent zero-call response; supported requests enter a bounded routed executor. On the 125-task public test split, the submitted configuration scored 98/125 (78.4%) in its available run while using 87.1k tokens and 9.58 seconds of successful LLM latency per task. The gain was concentrated in Hallucination, where the gate activated on 39/50 tasks and did not activate on Base or Disambiguation. Because the public split was used during development and only one final run was completed, we make no repeated-trial or hidden-set claim.

1 Problem and Development Strategy

Task and objective. CAR-bench evaluates consistency and limit-awareness in tool-using LLM agents [Kirmayr *et al.*, 2026]. The agent must act when an operation is supported and respond transparently when a required tool, parameter, or observation is unavailable. Aggressive action can hallucinate capabilities; broad refusal can miss solvable tasks. Track 2 allows at most 500k average tokens per task and five sequential LLM calls per baseline agent step.

Technical challenge. The central challenge is allocating inference where it is useful. A direct agent is fast but applies one policy everywhere. Planning, verification, and candidate selection may correct errors, but introduce new decisions, parsing surfaces, and latency. Long prompt-level rule sets can also perturb solved tasks. We therefore ask *which requests require more inference and which limits can be enforced without another model call.*

Development strategy. We used Cerebras-hosted `gpt-oss-120b` [OpenAI, 2025; Cerebras Systems, 2026] behind the A2A protocol [Agent2Agent Protocol, 2026]. Low latency enabled a promotion ladder from small diagnostics to the 125-task public test split. Generation and limit detection exhibited different failure modes: the model remained useful for supported tool reasoning, while a narrow request-conditioned check could reject structurally unsupported operations before each agent-generation step.

Contributions. Our report contributes (i) an experiment log showing why extra inference and global prompt optimization did not transfer reliably, (ii) a reproducible request-conditioned dependency gate that acts only on visible runtime evidence, and (iii) a bounded router-executor design with explicit call-depth and compute accounting.

1.1 Metrics and Promotion Gates

We report task-level reliability rather than pooled task-trial counts. Pass^1 is the expected success rate of one sampled run; $\text{Pass}@3$ is the fraction of tasks solved at least once in three attempts; and Pass^3 is the fraction solved in all three attempts. Pass^3 is the strict stability metric. For configurations evaluated once, only Pass^1 is reported.

Promotion used fixed gates. **Train15** (five training tasks per category) was the cheap diagnostic; **Holdout28** (10 Base, 10 Hallucination, 8 Disambiguation) was held out from prompt edits; **Near-holdout28** checked stability after training on 80 examples; **Public30** (10 tasks per category) tested transfer; and **Full125**, the complete 125-task public test split (50/50/25 by category), was reserved for promoted candidates. This sequence separated prompt development from public claims: gains had to transfer, preserve categories, and fit the compute budget. Weak transfer, instability, category collapse, or excess compute stopped promotion.

2 Experimental Results and Evolution

Table 1: Task-level results on the public test split. Pass@3 and Pass³ are reported only for three-run configurations. Compute is measured on a representative complete run; lower is better for tokens and latency.

System	Pass ¹ ↑	Pass@3 ↑	Pass ³ ↑	Tokens/task ↓	LLM s/task ↓	Evidence
Direct	58.7%	66.7%	44.7%	84.2k	5.26	3 runs
Planner	58.0%	69.3%	38.0%	123.9k	9.04	3 runs
Verifier	56.0%	69.3%	42.7%	161.9k	10.69	3 runs
Best-of-three	59.2%	—	—	213.2k	18.61	1 run
Guard	67.5%	78.4%	56.0%	105.5k	14.45	3 runs
Router + Guard	78.4%	—	—	87.1k	9.58	1 run

Table 2: Promotion ladder and design lessons. Each stage was retained only when its mechanism or evidence improved the next design decision.

Approach	Mechanism	Gate evidence	Decision and lesson
Direct / planner / verifier	One executor; plan-then-act; or execute-then-check	Three public test-split runs per system; Table 1	Extra passes did not improve stable accuracy; direct remained the efficient reference.
Context engineering	Examples and contracts for missing tools, unknown observations, argument schemas, and action discipline	Train15 11/15; Holdout28 23/28; Public30 14/30	Training gains did not transfer. Broad natural-language rules changed already-correct behavior.
Broader optimized context	One direct call with a prompt optimized from 80 diverse training examples	Train80 57/80; Near-holdout28 22/28; Public30 18/30 then 13/30	The repeat exposed instability; near-holdout performance was not sufficient for promotion.
GEPA/DSPy-style search	Offline trajectory reuse and reflective prompt merging [Khattab <i>et al.</i> , 2024; Agrawal <i>et al.</i> , 2026]	Existing-run oracle union 22/30; merged optimizer prompt 9/15	Retained atomic contracts, but rejected the merged prompt.
Best-of-three	Three independently generated actions followed by one selector call	Train80 60/80; Public30 17/30; Full125 74/125	Candidate multiplicity cost 213.2k tokens/task; weak candidate coverage, especially Hallucination, limited the selector.
Guard	Deterministic pre-route plus direct, contract-guard, and risk-reflection modes	Public30 19/30; primary Full125 85/125; repeated metrics in Table 1	Routing was promising; a counterfactual mechanism audit localized the most useful behavior to narrow deterministic checks.
Router + Guard	Request-conditioned dependency boundary around Guard	Train-Hallucination48 (complete train-Hall lane) 38/48; Full125 98/125	Selected submission: unsupported operations exit early; supported requests retain the routed executor.

2.1 Direct, Planner, and Verifier Baselines

Direct inference supplied the conversation and live tool schemas to one executor, which emitted either an action JSON object or a final response. Planner added a plan-generation call before execution; verifier added a post-generation review call that could revise the proposed action. Planning occasionally recovered a multi-step or preference-dependent case, but neither extra pass improved task-level stability. Planner increased Pass@3 while reducing Pass³, indicating more diverse rather than more reliable behavior. Verifier showed the same trade-off at still higher token cost. Direct therefore remained the reference path for later experiments.

2.2 Prompt and Context Optimization

Context engineering tested whether recurring benchmark contracts could be expressed directly in the executor prompt. Rules distinguished unavailable tools from unknown returned values, prohibited unsupported arguments, and mapped requests to relevant tool families. Train and holdout gains did not transfer to Public30. GEPA/DSPy-style trajectory reuse likewise found useful atomic contracts, but a merged optimizer prompt scored 9/15. We therefore retained the contracts and rejected global prompt expansion.

2.3 How Best-of-Three Worked

Best-of-three generated three candidate action JSON objects from the same conversation, live schemas, and observations; a fourth LLM call selected one for emission. This grounded selection in shared runtime evidence and kept sequential depth at four, similar in spirit to self-consistency [Wang *et al.*, 2023]. However, the selector could not recover when none of the generators produced a correct action. The full run used 26.65M

tokens (213.2k/task), scored 59.2% Pass¹, and reached 24/50 on Hallucination. Candidate coverage, rather than selection alone, was the bottleneck.

2.4 Promotion Interpretation

The small gates were filters, not headline evaluations. Train15 rejected cheap regressions; Holdout28 and Public30 tested transfer. Full125 was reserved for category-safe gains or narrowly testable mechanisms; repeated Public30 checks prevented unstable gains from reaching the costly split.

3 Router + Guard Method

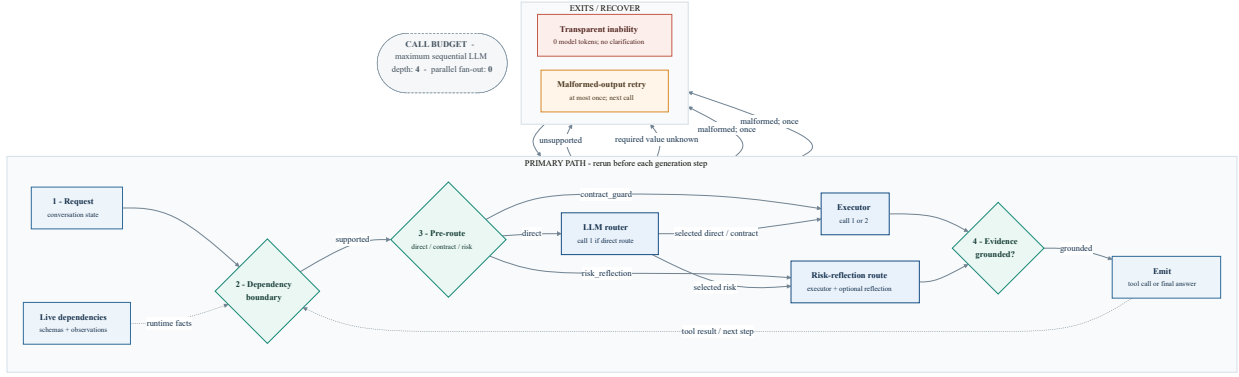


Figure 1: Final per-baseline-step architecture. Before each generation step, the dependency boundary reads the current request, live schemas, and accumulated observations. Contract/risk pre-routes may bypass the LLM router; the direct pre-route invokes it, and all selected modes share the bounded executor. Unsupported or unknown dependencies return a transparent zero-call response; the malformed-output retry is single-use, calls are sequential, and parallel fan-out is absent.

3.1 Overview

Router + Guard separates limit detection from supported-operation reasoning (Figure 1). Given a user request, live tool schemas, and accumulated tool results, the boundary is rerun before each agent-generation step. Unsupported operations produce a transparent inability response without invoking the model; supported requests enter a bounded route and one executor emits a tool call or final answer. Thus, a missing value discovered after a tool call can be handled at the next boundary check rather than being treated as a one-time pre-task decision.

3.2 Request-Conditioned Dependency Boundary

Motivation. Broad refusal rules are unsafe because a missing capability may be irrelevant to the current request. Conversely, allowing the model to discover every structural absence during generation spends tokens and permits fabricated workarounds. The dependency boundary therefore conditions every check on visible user intent.

Design. The boundary receives lower-cased text from the most recent 20 user messages, live function schemas, and recent tool results. Its implementation contains 22 hand-written trigger-based capability predicates covering climate, lighting, windows, sunroof, seats, charging/range, email/contact, navigation, route, and point-of-interest operations, plus 11 tool-specific unknown-result predicates. The predicates were written against the public tool schemas and generic request wording, with aliases expressed as explicit phrase triggers; development was informed by public failure analysis but never by task identifiers or expected actions.

Multiple predicates may be active; the source-order first missing dependency blocks the whole step. Otherwise the request passes to the router; semantic uncertainty and clarification remain the model’s responsibility.

Technical advantage. The check is auditable and zero-call. On the complete 48-task training Hallucination lane (**Train-Hallucination48**), Router + Guard scored 38/48, activated on 19 tasks, and passed all 19 guarded tasks; 14 used zero model tokens. Thirteen counterfactual tests varied relevant/irrelevant missing tools, parameters, evidence, and false/zero/empty values.

3.3 Bounded Routed Execution

Motivation. Supported requests still require semantic reasoning. Guard therefore handles cheap representability/evidence checks, while only passing requests receive expensive routed inference.

Design and advantage. The pre-route may bypass the LLM router for contract/risk modes; otherwise the router (threshold 0.72) selects direct, contract-guard, or risk-reflection. Each mode uses one `gpt-oss-120b` executor, with bounded reflection and one malformed-output retry. This preserves ReAct-style reasoning [Yao *et al.*, 2023] while bounding deliberation.

125-task public test-split result. Guard scored 85/125 (36/50, 36/50, 13/25); its three runs scored 85, 85, and 83. Router + Guard reached 98/125 (39/50, 45/50, 14/25) in one pass.

Stage	Input	Deterministic decision
Intent	Request text	22 trigger predicates; no match passes.
Schema	Live tools	Required or any-of tool exists.
Parameters	Schemas	Required top-level fields exist.
Evidence	Success result	11 relevance predicates; unknown blocks; false/0/empty pass.
Decision	Status	Fixed inability or pass; no partial execution or clarification.

4 Efficiency, Reliability, and Discussion

4.1 Call Depth and Compute Assumptions

Figure 1 gives the required call audit for one baseline agent step. The dependency boundary is rerun before each agent-generation step; a deterministic exit has depth zero. A direct route can bypass the LLM router, while the ordinary route uses router (depth 1), executor (2), at most one malformed-output retry (3), and optional risk reflection (4). There is no parallel LLM fan-out in the submitted system, and no retry loop can exceed one iteration.

Token figures include recorded input, reasoning, and output tokens exposed by the agent. Router + Guard used 10,883,233 tokens over 125 tasks (87,066/task), below the Track 2 allowance of 500k average tokens/task. Successful LLM latency was 1,197.3 seconds (9.58 seconds/task); end-to-end wall time was 2,899.8 seconds (23.20 seconds/task), including evaluator, A2A, tool, container, and network overhead. Guarded exits used neither model tokens nor model latency.

4.2 What the Experiment Ladder Shows

The comparison supports a selective-inference interpretation. Planning and verification increased average token use by approximately 47% and 92% relative to direct inference without improving Pass¹ or Pass³. Best-of-three consumed 2.53 times the direct token budget but could not recover when all candidates shared the same error. In contrast, Guard improved repeated reliability by assigning additional inference to riskier requests, and Router + Guard reduced average tokens relative to Guard by resolving clear unsupported dependencies before each generation step.

The result also clarifies why the deterministic boundary is narrow. Prompt and context optimization supplied useful local contracts, but broad rules changed the model’s behavior beyond their intended cases. Router + Guard converts only structural, request-conditioned contracts into deterministic checks; ambiguous semantic choices remain with the model. This division limits the deterministic surface while making its interventions testable.

4.3 Category-wise Behavior

The observed gain was concentrated in Hallucination, where the primary score rose from 36/50 for Guard to 45/50 for Router + Guard. The dependency boundary activated on 39/50 Hallucination tasks and all 39 passed; 23 exited before consuming agent-model tokens, while 16 activated only after model or tool evidence established a missing required value. It did not activate on Base or Disambiguation in the full run. Consequently, the observed Base (+3) and Disambiguation (+1) differences are not attributed to the boundary, while the Hallucination concentration matches its intended mechanism.

4.4 Limitations and Validity

We were not able to run three complete Router + Guard validation trials, so we do not report Router + Guard Pass@3 or Pass³. The available Guard stability reference is 56.0% Pass³ (and 78.4% Pass@3) from the preserved three-run record; Router + Guard’s 78.4% Pass¹ is one public pass and should be read as a candidate improvement on that Guard baseline,

not as a measured repeated-trial gain. The public promotion ladder was used repeatedly during development, so Full125 is evidence for candidate selection rather than a hidden generalization estimate. Gemini was the target evaluator, but when its rate limits interrupted local sweeps, GLM was used for affected local user/evaluator runs; the tables are therefore public development evidence, not official hidden-set measurements. Finally, Disambiguation remains the weakest lane at 14/25: persistent errors involve context-dependent ambiguity and follow-up resolution, and a train-only contract engine did not transfer.

5 Conclusion

Fast inference enabled broad experimentation, but selective inference produced the strongest submission. The final system combines a routed LLM executor with a narrow request-conditioned dependency boundary. It preserves model reasoning for supported operations, returns transparent zero-call responses for structurally unsupported operations, averages 87.1k tokens/task, and stays within a maximum sequential depth of four. Its 78.4% available one-pass public-test result is the submitted Track 2 measurement; repeated Router + Guard stability, hidden-set performance, and ranking remain unreported.

Acknowledgments

We thank the CAR-bench organizers for the Cerebras inference access and Codex subscription that supported the development and audit cycle. We also thank the organizers for making the public evaluation protocol and A2A tooling available; the resulting constraints shaped the selective-inference design reported here.

References

- [Agent2Agent Protocol, 2026] Agent2Agent Protocol. Agent2Agent protocol specification, 2026.
- [Agrawal *et al.*, 2026] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alex Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khat-tab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *The Fourteenth International Conference on Learning Representations*, 2026.
- [Cerebras Systems, 2026] Cerebras Systems. Cerebras Inference API documentation, 2026.
- [Khattab *et al.*, 2024] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. Dspy: Compiling declarative language model calls into self-improving pipelines. 2024.
- [Kirmayr *et al.*, 2026] Johannes Kirmayr, Lukas Stappen, and Elisabeth Andre. CAR-bench: Evaluating the consistency and limit-awareness of LLM agents under real-world uncertainty. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens, editors, *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 40599–40618, San Diego, California, United States, July 2026. Association for Computational Linguistics.
- [OpenAI, 2025] OpenAI. gpt-oss model card, 2025.
- [Wang *et al.*, 2023] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*, 2023.
- [Yao *et al.*, 2023] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023.